

Notes 5: Verification and Resampling Methods

Table of contents

Evaluation Metrics and the Confusion Matrix	60
Evaluation Metrics	60
Training and Test Sets	60
Overall Accuracy	61
The Confusion Matrix	63
Sensitivity and Specificity	64
Cross Validation with kNN Example	66
Cross Validation	66
K-Nearest Neighbors (kNN)	66
K-Nearest Neighbors (kNN): Example	67
Over-Training	69
Over-Smoothing	69
Picking the k in kNN	70
Mathematical Description of Cross Validation	71
K-Fold Cross Validation	72
Bootstrap	75
More on Bootstrapping	77
Bootstrap Example Using R:	80
More on the caret Package	82
The <code>caret train()</code> function	83
Cross Validation	83
Example: Fitting with Loess	86
Conclusion	87

Evaluation Metrics and the Confusion Matrix

Evaluation Metrics

Before we start describing approaches to optimize the way we build algorithms, we first need to define what we mean when we say one approach is better than another.

We use the `caret` package, which has several useful functions for building and assessing machine learning methods.

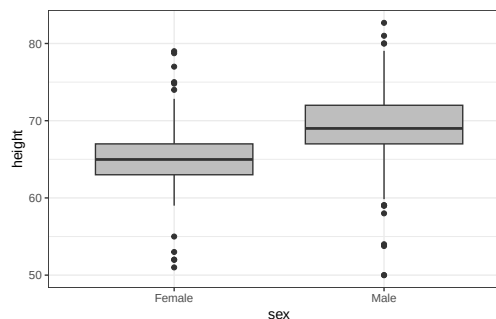
```
library(tidyverse)
library(caret)
```

For a first example, we use the height data in `dslabs`:

```
library(dslabs)
data(heights)
```

To summarize the data, consider the following boxplot:

```
ggplot(heights, aes(x = sex, y = height)) +
  geom_boxplot(fill = "gray") + theme_bw()
```



We will start with a simple example: suppose we want to predict sex using height. We start by defining the outcome and predictors.

```
y <- heights$sex
x <- heights$height
```

In this case, we have only one predictor, height, and `y` is clearly a categorical outcome since observed values are either `Male` or `Female`.

We know that we will not be able to predict Y very accurately based on X because male and female average heights are not that different relative to within group variability. But can we do better than guessing? To answer this question, we need a quantitative definition of better.

Training and Test Sets

Ultimately, a machine learning algorithm is evaluated on how it performs in the real world with completely new datasets. However, we usually have a dataset for which we know the outcomes, as we do with the heights: we know the sex of every student in our dataset. Therefore, to mimic the ultimate evaluation process, we typically split the data into two parts and act as if we don't know the outcome for one of these.

We refer to the group for which we know the outcome, and use to develop the algorithm, as the **training** set. We refer to the group for which we pretend we don't know the outcome as the **test** set.

`caret` includes the function `createDataPartition()` that helps us generate training and test sets:

```
set.seed(2026)
test_index <- createDataPartition(y, times = 1, p = 0.5, list = FALSE)
```

The argument `times` is used to define how many random samples of indexes to return, `p` is used to define what proportion of the data is represented by the index, and `list` is used to decide if we want the indexes returned as a list or not.

If `times = 1`, this is essentially the same as doing

```
set.seed(2026)
test_index <- sample(1:nrow(heights), size = round(nrow(heights) * 0.5))
```

We can use the result of the `createDataPartition()` function call to define the training and test sets like this:

```
test_set <- heights[test_index, ]
train_set <- heights[-test_index, ]
```

We develop an algorithm using **only** the training set. Once we are done developing the algorithm, we will **freeze** it and evaluate it using the test set. The simplest way to evaluate the algorithm when the outcomes are categorical is by simply reporting the proportion of cases that were correctly predicted in the test set. This metric is usually referred to as **overall accuracy**.

Overall Accuracy

To demonstrate the use of overall accuracy, we will build two competing algorithms and compare them.

Let's start by developing the simplest possible machine algorithm: guessing the outcome.

```
y_hat <- sample(c("Male", "Female"), nrow(test_set), replace = TRUE)
```

We are completely ignoring the predictor and simply guessing the sex.

In machine learning applications, it is useful to use factors to represent the categorical outcomes because **R** functions developed for machine learning, such as those in the `caret` package, require or recommend that categorical outcomes be coded as factors. So convert `y_hat` to factors using the `factor` function:

```
set.seed(2026)
y_hat <- sample(c("Male", "Female"), nrow(test_set), replace = TRUE) |>
  factor(levels = levels(test_set$sex))
```

The *overall accuracy* is simply defined as the overall proportion that is predicted correctly:

```
mean(y_hat == test_set$sex)
[1] 0.487619
```

Not surprisingly, our accuracy is about 50%. We are guessing!

Can we do better? Exploratory data analysis suggests we can because, on average, males are slightly taller than females:

```
heights |>
  group_by(sex) |>
  summarize(means = mean(height), sds = sd(height))
```

A tibble: 2 x 3

	sex	means	sds
	<fct>	<dbl>	<dbl>
1	Female	64.9	3.76
2	Male	69.3	3.61

But how do we make use of this insight? Let's try another simple approach: predict Male if height is within two standard deviations from the average male:

```
y_hat <- ifelse(x > 62, "Male", "Female") |>
  factor(levels = levels(test_set$sex))
```

The accuracy goes up from 0.50 to about 0.80:

```
mean(y == y_hat)
```

[1] 0.7933333

But can we do even better? In the example above, we used a cutoff of 62, but we can examine the accuracy obtained for other cutoffs and then pick the value that provides the best results.

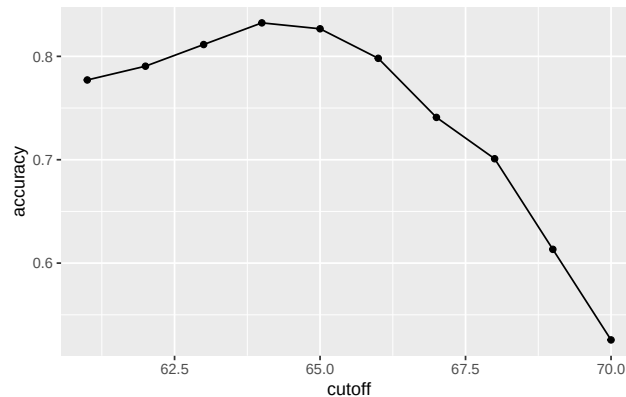
But remember, **it is important that we optimize the cutoff using only the training set:** the test set is only for evaluation.

Although for this simplistic example it is not much of a problem, later we will learn that evaluating an algorithm on the training set can lead to **overfitting**, which often results in dangerously over-optimistic assessments.

Here we examine the accuracy of 10 different cutoffs and pick the one yielding the best result:

```
cutoff <- seq(61, 70)
accuracy <- map_dbl(cutoff, function(x){
  y_hat <- ifelse(train_set$height > x, "Male", "Female") |>
    factor(levels = levels(test_set$sex))
  mean(y_hat == train_set$sex)
})
```

We can make a plot showing the accuracy obtained on the training set for males and females:



We see that the maximum value is:

```
max(accuracy)
[1] 0.832381
```

which is much higher than 0.5. The cutoff resulting in this accuracy is:

```
best_cutoff <- cutoff[which.max(accuracy)]
best_cutoff
[1] 64
```

We can now test this cutoff on our test set to make sure our accuracy is not overly optimistic:

```
y_hat <- ifelse(test_set$height > best_cutoff, "Male", "Female") |>
  factor(levels = levels(test_set$sex))
mean(y_hat == test_set$sex)
[1] 0.8209524
```

We see that it is similar to (actually slightly better than) the accuracy observed for the training set, but it is still better than guessing. And by testing on a dataset that we did not train on, we know our result is not due to cherry-picking a good result.

The prediction rule we developed in the previous section predicts **Male** if the student is taller than 64 inches. Given that the average female is about 64 inches, this prediction rule seems wrong. What happened? If a student is the height of the average female, shouldn't we predict **Female**?

The Confusion Matrix

Generally speaking, overall accuracy can be a deceptive measure. To see this, we will start by constructing what is referred to as the *confusion matrix*, which basically tabulates each combination of prediction and actual value. We can do this in **R** using the function `table`:

```
table(predicted = y_hat, actual = test_set$sex)
```

	actual	
predicted	Female	Male
Female	53	28
Male	66	378

If we study this table closely, it reveals a problem. If we compute the accuracy separately for each sex, we get:

	Actually Positive	Actually Negative
Predicted positive	True positives (TP)	False positives (FP)
Predicted negative	False negatives (FN)	True negatives (TN)

```
test_set |>
  mutate(y_hat = y_hat) |>
  group_by(sex) |>
  summarize(accuracy = mean(y_hat == sex))
```

```
# A tibble: 2 x 2
  sex      accuracy
<fct>    <dbl>
1 Female  0.445
2 Male    0.931
```

There is an imbalance in the accuracy for males and females: too many females are predicted to be male. We are calling almost half of the females male! How can our overall accuracy be so high then? This is because the **prevalence** of males in this dataset is high. These heights were collected from three data sciences courses, two of which had more males enrolled:

```
prev <- mean(y == "Male")
prev
```

```
[1] 0.7733333
```

So when computing overall accuracy, the high percentage of mistakes made for females is outweighed by the gains in correct calls for men. **This can actually be a big problem in machine learning.** If your training data is biased in some way, you are likely to develop algorithms that are biased as well. The fact that we used a test set does not matter because it is also derived from the original biased dataset. This is one of the reasons we look at metrics other than overall accuracy when evaluating a machine learning algorithm.

There are several metrics that we can use to evaluate an algorithm in a way that prevalence does not cloud our assessment, and these can all be derived from the confusion matrix. A general improvement to using overall accuracy is to study **sensitivity** and **specificity** separately.

Sensitivity and Specificity

In general, **sensitivity** is defined as the ability of an algorithm to predict a positive outcome when the actual outcome is positive: $\hat{Y} = 1$ when $Y = 1$. Stated in terms of probability, this is $P(\text{Predicted Positive} | \text{Actually Positive})$.

Because an algorithm that calls everything positive ($\hat{Y} = 1$ no matter what) has perfect sensitivity, this metric on its own is not enough to judge an algorithm.

For this reason, we also examine **specificity**, which is generally defined as the ability of an algorithm to not predict a positive $\hat{Y} = 0$ when the actual outcome is not a positive $Y = 0$. Stated in terms of probability, this is $P(\text{Predicted Negative} | \text{Actually Negative})$.

We name the four entries of the **confusion matrix**:

Sensitivity is typically quantified by $TP/(TP + FN)$, the proportion of actual positives (the first

column = $TP + FN$) that are called positives (TP). This quantity is also referred to as the **true positive rate** (TPR) or **recall**.

Specificity is defined as $TN/(TN + FP)$ or the proportion of negatives (the second column = $FP + TN$) that are called negatives (TN). This quantity is also called the **true negative rate** (TNR).

There is another way of quantifying accuracy which is $TP/(TP + FP)$ or the proportion of outcomes called positives (the first row or $TP + FP$) that are actually positives (TP). This quantity is referred to as **positive predictive value (PPV)** and also as **precision**. Note that, unlike TPR and TNR, precision depends on prevalence since higher prevalence implies you can get higher precision even when guessing.

The multiple names can be confusing, so we include a table to help us remember the terms. The table includes a column that shows the definition if we think of the proportions as probabilities.

Measure of	Name 1	Name 2	Definition	Prob. Representation
Sensitivity	TPR	Recall	$\frac{TP}{TP + FN}$	$P(\hat{Y} = 1 Y = 1)$
Specificity	TNR	$1 - \text{FPR}$	$\frac{TN}{TN + FP}$	$P(\hat{Y} = 0 Y = 0)$
Precision	PPV		$\frac{TP}{TP + FP}$	$P(Y = 1 \hat{Y} = 1)$
	NPV		$\frac{TN}{TN + FN}$	$P(Y = 0 \hat{Y} = 0)$

Here TPR is True Positive Rate, FPR is False Positive Rate, PPV is Positive Predictive Value, and NPV is Negative Predictive Value.

The `caret` package has the function `confusionMatrix()` that computes the confusion matrix for us. The function expects factors as input, and the first level is considered the positive outcome or $Y = 1$. In our example, `Female` is the first level because it comes before `Male` alphabetically. It can be helpful and avoid issues to explicitly tell the function which category we are predicting using the `positive` argument.

```
cm <- confusionMatrix(data = y_hat, reference = test_set$sex,
                      positive = "Female")
cm$table
```

```
      Reference
Prediction Female Male
  Female      53   28
  Male       66  378
```

You can access these directly, for example, like this:

```
cm$overall["Accuracy"]
Accuracy
0.8209524

cm$byClass[c("Sensitivity", "Specificity", "Prevalence")]
Sensitivity Specificity Prevalence
0.4453782   0.9310345   0.2266667
```

```
cm$byClass["Balanced Accuracy"]
Balanced Accuracy
0.6882063
```

The balanced accuracy is calculated by taking the average of sensitivity and specificity.

We can see that the high overall accuracy is possible despite relatively low sensitivity. As we hinted at above, the reason this happens is because of the low prevalence (0.23): the proportion of females is low. Because prevalence is low, failing to predict actual females as females (low sensitivity) does not lower the accuracy as much as failing to predict actual males as males (low specificity).

Let's increase the cutoff to 67 inches.

```
y_hat2 <- ifelse(test_set$height > 67, "Male", "Female") |>
  factor()
cm2 <- confusionMatrix(data=y_hat2, reference=test_set$sex)
cm2$overall["Accuracy"]
Accuracy
0.7333333

cm2$byClass[c("Sensitivity", "Specificity", "Prevalence")]
Sensitivity Specificity Prevalence
0.7899160 0.7167488 0.2266667

cm2$byClass["Balanced Accuracy"]
Balanced Accuracy
0.7533324
```

Cross Validation with kNN Example

Cross Validation

In the previous example, we split our data into training and testing groups so as not to overfit and to get an idea of how the model works for new data.

Cross-validation takes this one step forward. Cross-validation is one of the most important ideas in machine learning. Here we focus on the conceptual and mathematical aspects. We will describe how to implement cross validation in practice in later examples.

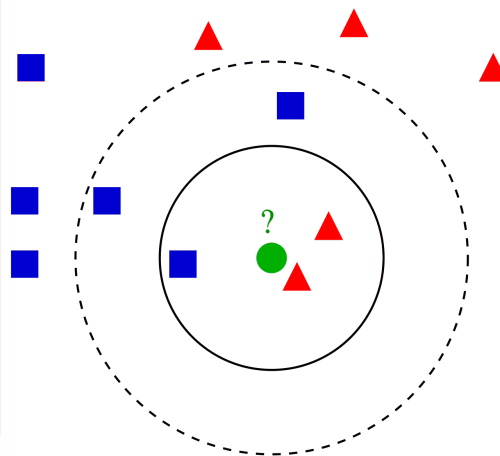
K-Nearest Neighbors (kNN)

To motivate the concept, let's consider an actual machine learning algorithm: **k-nearest neighbors (kNN)**.

K-nearest neighbor algorithm pseudocode

Programming languages like Python and R are used to implement the KNN algorithm. The following is the pseudocode for KNN:

1. Load the data
2. Choose K value
3. For each data point in the data:
 - Find the Euclidean distance to all training data samples
 - Store the distances on an ordered list and sort it
 - Choose the top K entries from the sorted list
 - Label the test point based on the majority of classes present in the selected points
4. End



With k-nearest neighbors (kNN) we estimate the conditional probability function:

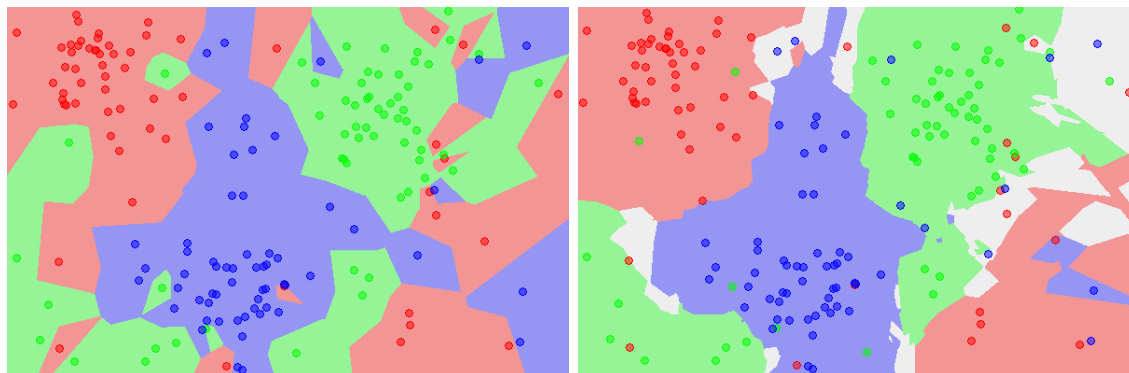
$$p(x_1, x_2) = P(Y = 1 \mid X_1 = x_1, X_2 = x_2).$$

First we define the distance between each of the observations. Then, for any point (x_1, x_2) for which we want an estimate of $p(x_1, x_2)$, we look for the k nearest points to (x_1, x_2) and then take an average of the 0s and 1s associated with these points. We refer to the set of points used to compute the average as the *neighborhood*. This gives us an estimate $\hat{p}(x_1, x_2)$.

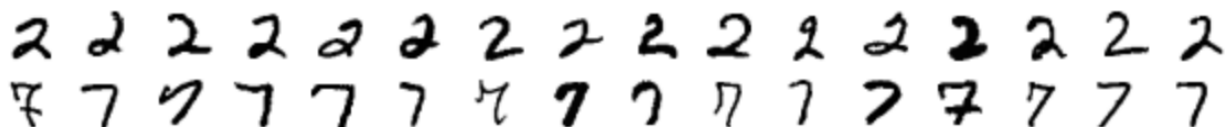
Using $k = 1$:

and

$k = 5$:

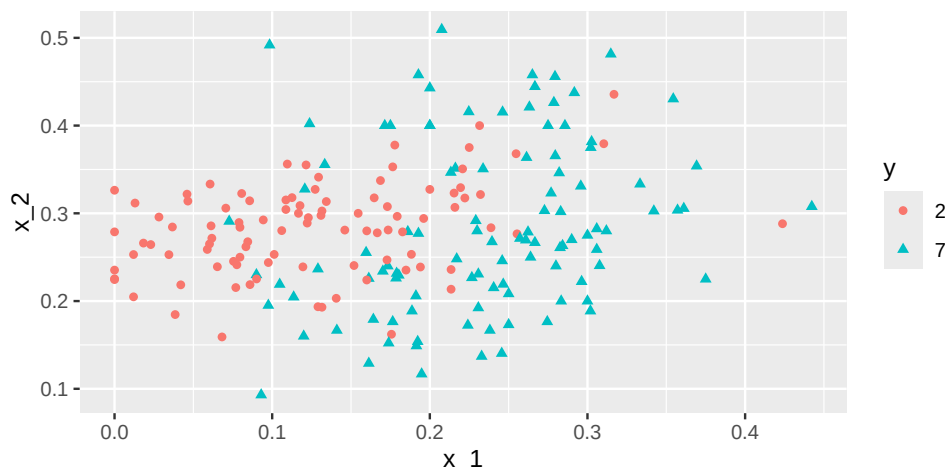
**K-Nearest Neighbors (kNN): Example**

We will use the `mnist_27` data set in the `dslabs` package. This dataset contains information about predicting hand-written numbers. In this dataset, only 2's and 7's are considered. The data have already been split into a testing and training set. Each of these have three columns: `y`, `x_1`, and `x_2`. `y` is the true number, `x_1` is the proportion of dark pixels in the upper-left quadrant, and `x_2` is the proportion of dark pixels in the lower-right quadrant.



Using this data set:

```
library(dslabs)
mnist_27$test |>
  ggplot(aes(x_1, x_2, color = y, shape = y)) +
    geom_point()
```



We can use the `knn3` function from the `caret` package as follows, with the number of neighbors equal to $k = 5$:

```
library(caret) |> suppressPackageStartupMessages()
knn_fit <- knn3(y ~ ., data = mnist_27$train, k = 5)
```

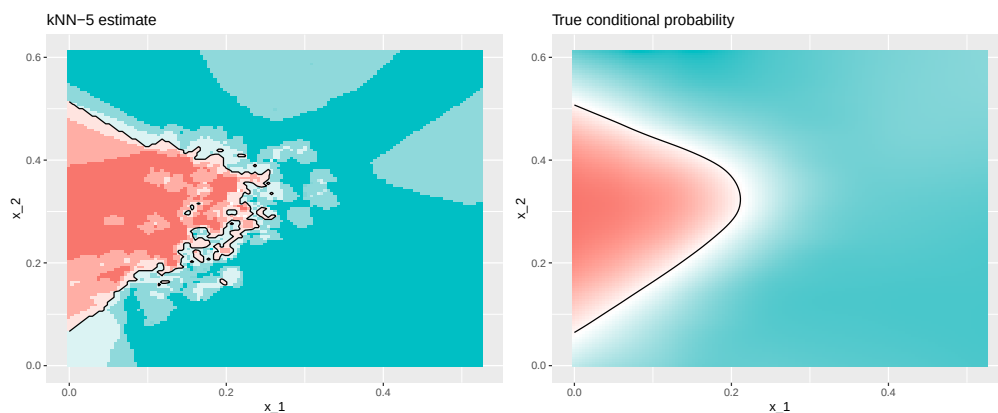
Note that we have two classes here: a 2 or a 7. Therefore, we should use an odd number for k to avoid ties. If we had an odd number of classes, then an even number for k should be used.

Since our data set is balanced (the prevalence is 0.53) and we care just as much about sensitivity as we do about specificity, we will use accuracy to quantify performance.

The `predict` function for `knn` produces a probability for each class.

```
y_hat_knn <- predict(knn_fit, mnist_27$test, type = "class")
confusionMatrix(y_hat_knn, mnist_27$test$y)$overall["Accuracy"]
```

Accuracy
0.815



Over-Training

Over-training or **over-fitting** results in having higher accuracy in the train set compared to the test set:

```
y_hat_knn <- predict(knn_fit, mnist_27$train, type = "class")
confusionMatrix(y_hat_knn, mnist_27$train$y)$overall["Accuracy"]
Accuracy
0.86
```

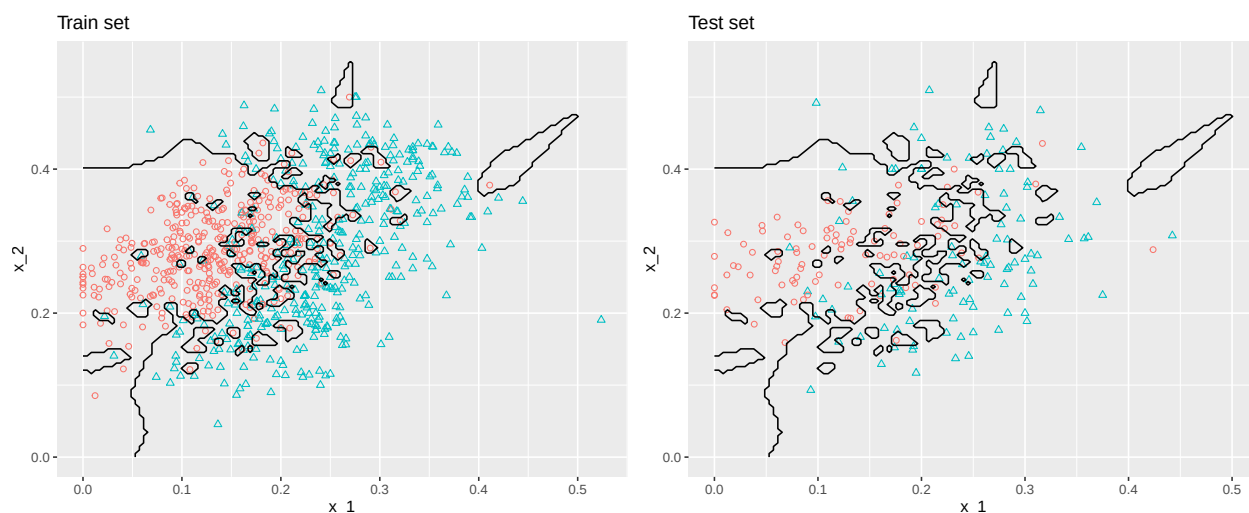
```
y_hat_knn <- predict(knn_fit, mnist_27$test, type = "class")
confusionMatrix(y_hat_knn, mnist_27$test$y)$overall["Accuracy"]
Accuracy
0.815
```

Over-training is at its worst when we set $k = 1$:

```
knn_fit_1 <- knn3(y ~ ., data = mnist_27$train, k = 1)
y_hat_knn_1 <- predict(knn_fit_1, mnist_27$train, type = "class")
confusionMatrix(y_hat_knn_1, mnist_27$train$y)$overall["Accuracy"]
Accuracy
0.99375
```

```
y_hat_knn_1 <- predict(knn_fit_1, mnist_27$test, type = "class")
confusionMatrix(y_hat_knn_1, mnist_27$test$y)$overall["Accuracy"]
Accuracy
0.81
```

We can see the over-fitting problem in this figure.



Over-Smoothing

Although not as badly as with $k = 1$, we saw that $k = 5$ is also over-trained. Hence, we should consider a larger k . Let's try, as an example, a much larger number: $k = 401$.

```
knn_fit_401 <- knn3(y ~ ., data = mnist_27$train, k = 401)
y_hat_knn_401 <- predict(knn_fit_401, mnist_27$train, type = "class")
confusionMatrix(y_hat_knn_401, mnist_27$train$y)$overall["Accuracy"]
```

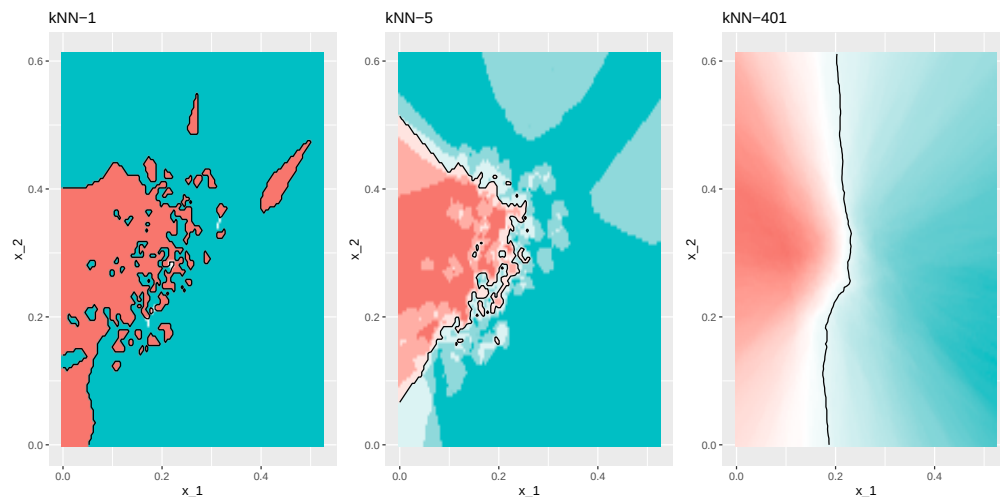
Accuracy

0.78625

```
y_hat_knn_401 <- predict(knn_fit_401,mnist_27$test,type = "class")
confusionMatrix(y_hat_knn_401, mnist_27$test$y)$overall["Accuracy"]
```

Accuracy

0.76



Picking the k in kNN

So how do we pick k ? In principle, we want to pick the k that maximizes accuracy, or minimizes the expected MSE (defined later).

The goal of **cross validation** is to estimate these quantities for any given algorithm and set of tuning parameters such as k . To understand why we need a special method to do this let's repeat what we did above but for different values of k :

```
ks <- seq(3, 251, 2)
```

We do this using `map_df` function to repeat the function for each element of the vector.

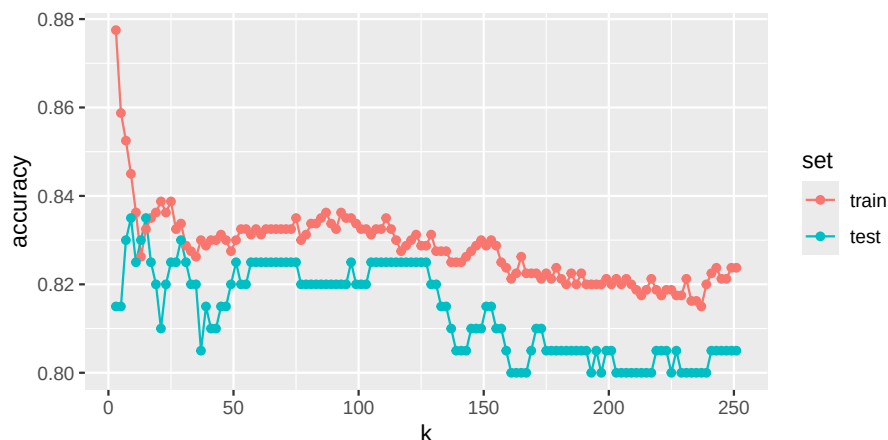
```
library(purrr)
accuracy <- map_df(ks, function(k){
  fit <- knn3(y ~ ., data = mnist_27$train, k = k)

  y_hat <- predict(fit, mnist_27$train, type = "class")
  cm_train <- confusionMatrix(y_hat, mnist_27$train$y)
  train_acc <- cm_train$overall["Accuracy"]

  y_hat <- predict(fit, mnist_27$test, type = "class")
  cm_test <- confusionMatrix(y_hat, mnist_27$test$y)
  test_acc <- cm_test$overall["Accuracy"]

  tibble(train = train_acc, test = test_acc)
})
```

Note that we estimate accuracy by using both the training set and the test set. We can now plot the accuracy estimates for each value of k :



If we were to use these estimates to pick the k that maximizes accuracy, we would use the estimates built on the test data:

```
ks[which.max(accuracy$test)]
```

```
[1] 9
```

```
max(accuracy$test)
```

```
[1] 0.835
```

First, note that the “best” estimate for k obtained for the training set is generally lower than the estimate obtained using the test set, with the difference larger for smaller values of k . This is due to over-training.

Also note that the accuracy versus k plot is quite jagged. We do not expect this because small changes in k should not affect the algorithm’s performance too much. The jaggedness is explained by the fact that the accuracy is computed on a sample and therefore is a random variable. This demonstrates why we prefer to minimize the expected, or average, loss (defined later) rather than the loss we observe with one dataset.

Another reason we need a better estimate of accuracy is that if we use the test set to pick this k , we should not expect the accompanying accuracy estimate to extrapolate to the real world. This is because even here we broke a golden rule of machine learning: we selected the k using the test set. Cross validation also provides an estimate that takes this into account.

Mathematical Description of Cross Validation

A common goal of machine learning is to find an algorithm that produces predictors $\hat{Y}$ for an outcome Y that minimizes the mean squared error (MSE):

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (Y_i - \hat{Y}_i)^2$$

When all we have at our disposal is one data set, we can estimate the MSE with the observed MSE like this:

$$\hat{\text{MSE}} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

These two are often referred to as the *true error* and *apparent error*, respectively.

There are two important characteristics of the apparent error we should always keep in mind:

1. Because our data is random, the apparent error is a random variable. For example, the dataset we have may be a random sample from a larger population. An algorithm may have a lower apparent error than another algorithm due to luck.
2. If we train an algorithm on the same dataset that we use to compute the apparent error, we might be overtraining. In general, when we do this, the apparent error will be an underestimate of the true error. We will see an extreme example of this with k-nearest neighbors.

Cross validation is a technique that permits us to alleviate both these problems. To understand cross validation, it helps to think of the true error, a theoretical quantity, as the average of many apparent errors obtained by applying the algorithm to B new random samples of the data, none of them used to train the algorithm:

$$\frac{1}{B} \sum_{b=1}^B \frac{1}{n} \sum_{i=1}^n (y_i^b - \hat{y}_i^b)^2$$

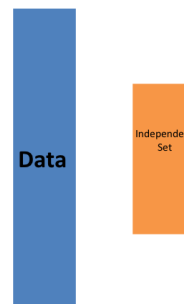
with B a large number that can be thought of as practically infinite.

As already mentioned, this is a theoretical quantity because we only have available one set of outcomes: $y_1, \dots, y_n$. Cross validation is based on the idea of imitating the theoretical setup above as best we can with the data we have.

To do this, we have to generate a series of different random samples. There are several approaches we can use, but the general idea for all of them is to randomly generate smaller datasets that are not used for training, and instead used to estimate the true error.

K-Fold Cross Validation

The first one we describe is **K-fold cross validation**. Note that this K is different than the k -nearest neighbors k value. A machine learning challenge starts with a data set (blue). We need to use this to build an algorithm that will be used in an independent validation data set (orange).



But we don't get to see these independent datasets.

So to imitate this situation, we carve out a piece of our dataset and pretend it is an independent dataset: we divide the dataset into a **training set** (blue) and a **test set** (red). We will train our algorithm exclusively on the training set and use the test set only for evaluation purposes.



- We usually try to select a small piece of the data set so that we have as much data as possible to train. However, we also want the test set to be large enough so that we obtain a stable estimate of the loss without fitting an impractical number of models. Typical choices are to use 10%-20% of the data for testing.
- Now this presents a new problem because for most machine learning algorithms we need to select parameters, for example the number of neighbors k in k-nearest neighbors. Here, we will refer to the set of parameters as λ .
- We need to optimize algorithm parameters without using our test set and we know that if we optimize and evaluate on the same dataset, we will overtrain.
- For each set of algorithm parameters being considered, we want an estimate of the MSE and then we will choose the parameters with the smallest MSE. Cross validation provides this estimate.

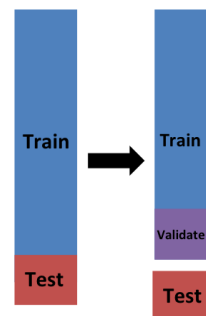
First, before we start the cross validation procedure, it is important to fix all the algorithm parameters. We will use $\hat{y}_i(\lambda)$ to denote the predictors obtained when we use parameters λ .

So, if we are going to imitate this definition:

$$\text{MSE}(\lambda) = \frac{1}{B} \sum_{b=1}^B \frac{1}{n} \sum_{i=1}^n \left(y_i^b - \hat{y}_i^b(\lambda) \right)^2$$

We want to consider datasets that can be thought of as an independent random sample and we want to do this several times. With K-fold cross validation, we do it K times. We are showing an example that uses $K = 5$.

We will eventually end up with K samples, but let's start by describing how to construct the first: we simply pick $m = n/K$ observations at random (we round if m is not a round number) and think of these as a random sample $y_1^b, \dots, y_m^b$, with $b = 1$. We call this the validation set:

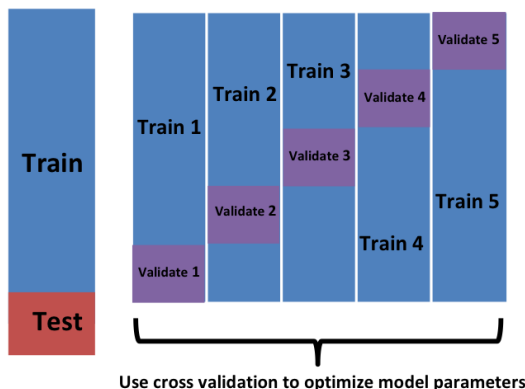


Now we can fit the model in the training set, then compute the apparent error on the validation set:

$$\text{MSE}_b(\boldsymbol{\lambda}) = \frac{1}{m} \sum_{i=1}^m \left(y_i^b - \hat{y}_i^b(\boldsymbol{\lambda}) \right)^2$$

Note: n = sample size of training and testing data sets combined while m = sample size of validation data set.

Note that this is just one sample and will therefore return a noisy estimate of the true error. This is why we take K samples, not just one. In K -fold cross validation, we randomly split the observations into K non-overlapping sets:



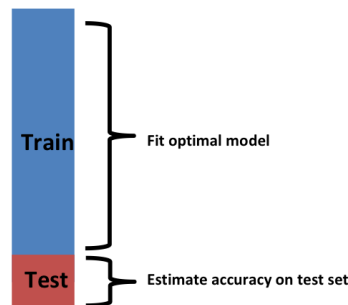
Now we repeat the calculation above for each of these sets $b = 1, \dots, K$ and obtain $\text{MSE}_1(\boldsymbol{\lambda}), \dots, \text{MSE}_K(\boldsymbol{\lambda})$. Then, for our final estimate, we compute the average:

$$\text{MSE}(\boldsymbol{\lambda}) = \frac{1}{K} \sum_{b=1}^K \text{MSE}_b(\boldsymbol{\lambda}) = \frac{1}{K} \sum_{b=1}^K \left[\frac{1}{m} \sum_{i=1}^m \left(y_i^b - \hat{y}_i^b(\boldsymbol{\lambda}) \right)^2 \right]$$

and obtain an estimate of our loss.

The final step would be to select the $\boldsymbol{\lambda}$ that minimizes the MSE.

We have described how to use cross validation to optimize parameters. However, we now have to take into account the fact that the optimization occurred on the training data. Here is where we use the test set we separated early on:



Now how do we pick the cross validation K (not the k for kNN)? Large values of K are preferable because the training data better imitates the original dataset. However, larger values of K will have much slower computation time: for example, 100-fold cross validation will be 10 times slower than 10-fold cross validation. For this reason, the choices of $K = 5$ and $K = 10$ are popular.

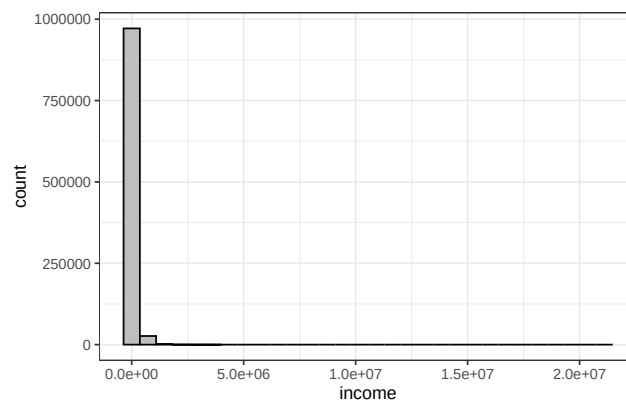
One way we can improve the variance of our final estimate is to take more samples. To do this, we would no longer require the training set to be partitioned into non-overlapping sets. Instead, we would just pick K sets of some size at random.

One popular variation of this technique, at each fold, picks observations at random with replacement (which means the same observation can appear twice). This approach has some advantages and is generally referred to as the *bootstrap*.

Bootstrap

Suppose the income distribution of your population is as follows:

```
set.seed(2026)
N <- 10^6 # Population size
income = 10^(rnorm(N, log10(45000), log10(3)))
ggplot(data.frame(income = income), aes(x = income)) +
  geom_histogram(bins = 30, color = "black", fill = "gray") + theme_bw()
```



The population median is:

```
M <- median(income) # True median
M
[1] 44946.33
```

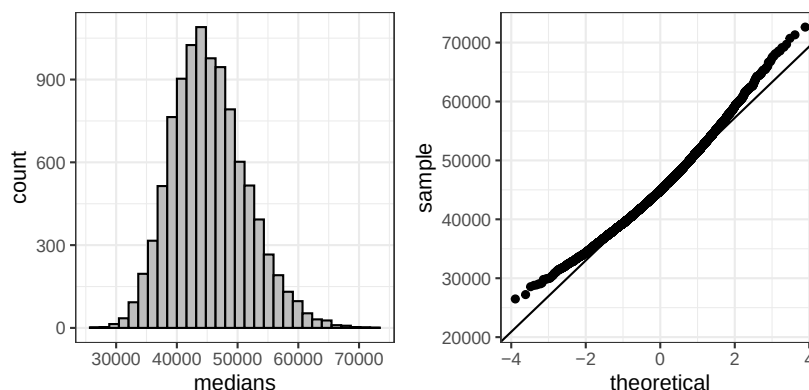
Suppose we don't have access to the entire population, but want to estimate the true median M . We take a sample of 100 and estimate the population median M with the sample median m :

```
n <- 100 # Sample size
samp <- sample(income, n)
median(samp)
[1] 41893.87
```

Can we construct a confidence interval? What is the distribution of m ?

We can use a **Monte Carlo** simulation to learn the distribution of m . A Monte Carlo simulation consists of simulating many samples from the population, computing the statistic of interest on each sample, and then using the resulting values to approximate the distribution of the statistic. Here is how we can do this for the median:

```
library(patchwork)
nsims <- 10^4 # Number of simulated samples
medians <- replicate(nsims, {X <- sample(income, n); median(X)})
p1 <- ggplot(data.frame(medians = medians), aes(x = medians)) +
  geom_histogram(bins = 30, color = "black", fill = "gray") +
  theme_bw()
p2 <- ggplot(data.frame(medians = medians), aes(sample = medians)) +
  geom_qq_line() +
  geom_qq() +
  theme_bw()
p1 + p2
```



If we know this distribution, we can construct a confidence interval. The problem here is that, as we have already described, in practice we do not have access to the distribution. We can see that the 95% confidence interval based on CLT, given here:

```
lower <- median(samp) - 1.96 * sd(samp) / sqrt(n)
upper <- median(samp) + 1.96 * sd(samp) / sqrt(n)
c(lower, upper)
[1] 19631.66 64156.09
```

is quite different from the confidence interval we would generate if we know the actual distribution of m :

```
quantile(medians, c(0.025, 0.975))
      2.5%      97.5%
34539.07 58794.24
```

Why are the intervals so different?

The bootstrap permits us to approximate a Monte Carlo simulation without access to the entire distribution. The general idea is relatively simple. We act as if the observed sample is the population. We then sample (with replacement) datasets, of the same sample size as the original dataset. Then we compute the summary statistic, in this case the median, on these *bootstrap samples*.

Theory tells us that, in many situations, the distribution of the statistics obtained with bootstrap samples approximate the distribution of our actual statistic.

This is how we construct bootstrap samples and an approximate distribution:

```
B <- 10^4 # Number of bootstrap samples
m_star <- replicate(
  B,
  {
    X_star <- sample(income, n, replace = TRUE)
    median(X_star)
  }
)
```

Note a confidence interval constructed with the bootstrap is much closer to one constructed with the theoretical distribution:

```
quantile(m_star, c(0.025, 0.975))
      2.5%      97.5%
34558.60 58612.31
```

More on Bootstrapping

- When did bootstrapping originate? The first paper on the theory of bootstrapping is attributed to Bradley Efron (1979), but it wasn't until the early 1990's with the *Science* paper by Efron & Tibshirani (1991), their subsequent book (1993), and advances in modern computing that the method gained widespread attention and use. It is now one of the most common methods of obtaining measures of uncertainty in numerous statistical applications where no closed-form expressions and/or approximations are available.
- The idea of resampling your data to generate more data is akin to “pulling yourself up by your bootstraps” to do analysis without extra outside help. Statisticians were very suspicious of bootstrapping when it first gained attention, but are now convinced of its power. If it seems like you are getting something for nothing (or downright cheating) with bootstrapping, I tend to agree, but the method has proven useful in an incredible variety of problems (censored data, survival data, time series, classification trees, goodness of fit statistics, linear and nonlinear regression, etc.).

Suppose $x_1, \dots, x_n$ is a random sample from some large population and we wish to estimate some population parameter θ . If $\hat{\theta}$ is an estimate of θ with some estimated standard error $SE(\hat{\theta})$, then under certain conditions a $100(1 - \alpha)\%$ confidence interval for θ can be formed as:

$$\hat{\theta} \pm t_{\alpha/2} SE(\hat{\theta}) \quad (z_{\alpha/2} \text{ may sometimes be used instead of } t_{\alpha/2}).$$

- For example, if $\theta = \mu$ is the population mean, then a $100(1 - \alpha)\%$ confidence interval for μ is given by:

$$\bar{x}_n \pm t_{\alpha/2} \cdot \frac{s}{\sqrt{n}} \Leftrightarrow (\text{estimate}) \pm \left(\begin{array}{c} \text{critical} \\ \text{value} \end{array} \right) \cdot \left(\begin{array}{c} \text{standard error} \\ \text{of the estimate} \end{array} \right).$$

The validity and accuracy of this CI depends on three basic assumptions:

1. $SE(\hat{\theta})$ is a good estimate of the standard deviation of the sampling distribution of $\hat{\theta}$.
2. $\hat{\theta}$ is an unbiased or nearly unbiased estimator for the parameter θ .

3. The sampling distribution of $\hat{\theta}$ is approximately normal. (The Central Limit Theorem can often be used here.)

The method of bootstrapping can address all of these issues:

1. It can provide an estimate of $SE(\hat{\theta})$ when no theoretical closed-form expression for $SE(\hat{\theta})$ exists, or provide an alternative if we are uncertain about the accuracy of an existing estimate. Can you think of some examples where the SE cannot be estimated or must be approximated?
2. It can provide an estimate of the bias of $\hat{\theta}$ as an estimator of θ .
3. It can provide information on the shape of the sampling distribution of $\hat{\theta}$. This can be used to calculate improved confidence intervals over the normal-based ones if the sampling distribution is not normal.

So what is bootstrapping? The theory behind bootstrapping can best be explained with a simple example.

- Suppose we have a simple random sample (SRS) and wish to estimate the standard error of the sample 10% trimmed mean, say $\bar{x}_{TM}$, as an estimate of the population mean, say μ . Unlike the sample mean, whose variance depends only on the population variance ($\sigma/\sqrt{n}$), $Var(\bar{x}_{TM})$ depends on the exact population distribution, which of course we don't know.
- The idea behind the (nonparametric) bootstrap is this: if we knew the x -values for the entire population ($x_1, \dots, x_N$), then we could estimate the sampling distribution of $\bar{x}_{TM}$ by simulation. How?
 - 1.
 - 2.
 - 3.

We could estimate the $SE(\bar{x}_{TM})$ from the standard deviation of the several thousand trimmed means.

- Unfortunately, we do not know the x -values for the population; we only know the x -values for our sample. Bootstrapping says to assume that the population looks exactly like our sample, only many times larger. This is really our best nonparametric guess as to what the population looks like.
- For example, suppose the following SRS of x -values ($n = 20$) represents guesses of my age when I taught a stats class in 2017:

26 25 26 29 29 28 27 29 28 26 41 33 29 34 27 33 27 30 29 30

then we assume that 5% of the population x -values are 26, 5% are 25, etc. In this way, this “bootstrap population” represents our best guess as to what the actual population looks like.

- To perform bootstrapping then, we simulate drawing samples of size $n = 20$ from this “bootstrap population.” If the population size N is large relative to n , this is equivalent to

drawing random samples of size 20 with replacement from the original sample of size 20. Since we are sampling with replacement, we will not necessarily get the same sample every time. For example, if `ages` is the vector of 20 age guesses, issuing the **R** command `sample(ages, 20, replace = T)` two times gave the following two samples:

```
29 27 34 25 27 29 33 28 25 28 26 29 26 26 27 26 27 28 29 28
28 29 33 27 26 27 26 29 33 33 29 29 26 30 26 30 30 27 25 26
```

- Finally, generate a large number of these bootstrap samples (say 10,000 or more), calculate the sample 10% trimmed mean for each sample, and then calculate the sample standard deviation of the 10,000 sample trimmed means as an estimate of $SE(\bar{x}_{TM})$.

Example: Finding the $SE(\bar{x}_{TM})$ using R: Suppose as discussed above that an SRS of size 20 yielded the age guesses 26, 25, ..., 30, and we would like to estimate $SE(\bar{x}_{TM})$. To do this in **R**, we could issue the commands below, and consider the output which follows. There is a command in **R** called `boot()` that will do this automatically, but to understand how to use it, it's useful to see how we might write our own program to do bootstrapping in **R**. To generate one bootstrap sample, we could do the following:

```
set.seed(2026)
ages <- c(26,25,26,29,29,28,27,29,28,26,41,33,29,34,27,33,27,30,29,30)
i <- sample(1:20, 20, replace = T)
i
[1] 1 6 13 15 12 4 16 5 12 2 19 15 14 17 12 10 18 2 5 12
ages[i]
[1] 26 28 29 27 33 29 33 29 33 25 29 27 34 27 33 26 30 25 29 33
mean(ages[i], trim = 0.10)
[1] 29.25
```

We would want to store the value of the trimmed mean for the bootstrap sample (the “29.25”) and then repeat the process (with a loop) several thousand times, getting a new vector `i` each time we called “sample”. Notice that rather than sampling with replacement from the data vector (e.g., `sample(ages, 20, replace = T)`), I sampled from the integers 1 to n and then got the data values for the bootstrap sample by using the expression `x[i]`. There is no difference between the two ways of doing it, but this way illustrates how the `bootstrapping()` function does it in our `math3190package` and will help in understanding how to use `bootstrapping`. The commands to carry out the bootstrapping look like this.

```
library(math3190package) |> suppressPackageStartupMessages()
set.seed(2026)
ages <- c(26,25,26,29,29,28,27,29,28,26,41,33,29,34,27,33,27,30,29,30)
trmean <- function(x,i) {mean(x[i], trim = 0.10)}
(b1 = bootstrapping(ages, trmean, 10000))
# A tibble: 1 x 5
  statistic sample_average original_value bias std.error
  <chr>          <dbl>          <dbl> <dbl>    <dbl>
1 stat1          28.8          28.8 0.0769   0.733
```

- From this output, we have the average of the 10,000 bootstrapped trimmed means is 28.83, the trimmed mean value from our original sample is 28.75, so estimated bias = sample_average - original_value = 0.08. Then the bootstrap estimated standard error: $SE_B(\bar{x}_{TM}) = 0.733$.
- The estimated bias is thus the difference between the average of the 10,000 bootstrap trimmed means and the actual sample trimmed mean.
- **If** (not usually the case) the sampling distribution of the sample trimmed mean is approximately normal, we could establish an approximate 95% confidence interval for μ_{TM} as:

$$\bar{x}_{TM} \pm z_{.025} \cdot SE(\bar{x}_{TM}) = 28.75 \pm 1.96(0.733) = 28.75 \pm 1.44 \implies 27.3 < \mu_{TM} < 30.2.$$

In the bootstrap procedure outlined above, it is important to note the following assumptions and related issues:

1. It assumes an infinite population since it uses sampling with replacement from the observed sample. This is not a problem if the population size N is in fact large relative to the sample size n . However, if N is small, it is unclear what to do!
2. It assumes simple random sampling. Since the bootstrap samples are generated by simple random sampling (from an infinite population), it is estimating the sampling distribution of $\hat{\theta}$ for simple random sampling from an infinite population.

If the original sample was acquired based on some other type of sampling scheme, such as stratified random sampling, then we want to estimate the sampling distribution of $\hat{\theta}$ for a *stratified random sample*. Hence, the bootstrap samples must be taken in exactly the same way as the original sample. It is sometimes very tricky to figure out how to do this correctly and is still an active area of research for many types of sampling.

Bootstrap Example Using R:

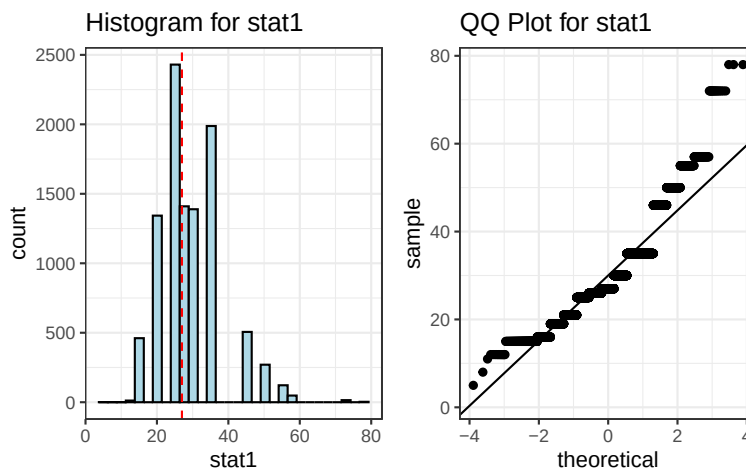
Consider the data on grocery costs at a particular grocery, given below. In class, we learned how to develop a confidence interval for M using percentiles from a binomial distribution with the `SignTest()` function in **R**. Here, we use bootstrapping to estimate the SE of the sample median $m = \widehat{M}$, find a confidence interval for M , and examine the sampling distribution of m .

1	3	4	5	5	8	11	12	15	15	16	19	21	25	26	27
30	35	35	46	50	55	57	72	78	85	93	137	158	212	269	

Taking 10,000 bootstrap samples from our sample of 31 costs is performed as follows:

```
set.seed(2026)
spent <- c(1,3,4,5,5,8,11,12,15,15,16,19, # Defines the 31 grocery amounts
          21,25,26,27,30,35,35,46,50,55, # spent, as given in the notes
          57,72,78,85,93,137,158,212,269)
med <- function(x,i) median(x[i]) # Defines "med" as the median function
(b2 <- bootstrapping(spent, med, 10000)) # Bootstraps the median of spent data
# A tibble: 1 x 5
  statistic sample_average original_value bias std.error
  <chr>          <dbl>          <dbl> <dbl>    <dbl>
1 stat1          29.3            27    2.35    8.48
```

From this output, we have the sample median $m = 27$, the estimated bias $= 2.35$, the mean of the bootstrap medians is 29.3, and the bootstrap estimated standard error: $SE_B(m) = 8.48$. To use a normal-based confidence interval, we need to assess whether the sampling distribution of the sample median is approximately normal. We can do this by examining the distribution of the 10,000 bootstrap medians, by issuing the command `plot(b2)`. This command creates the following two displays. Do the medians appear normal?



Instead of relying on the approximate normality of the bootstrap median distribution to construct a 95% confidence interval, we could compute the 2.5 and 97.5 percentiles for the bootstrap sample, and use these as the limits of the CI. This can adjust for any bias or skewness that might be present in the distribution. One of the methods built into **R** is the “bias-corrected & adjusted” (BCa) method, whereby relevant percentiles of the bootstrap sample are estimated. To illustrate this:

```
boot_ci(b2)      # Constructs bootstrap CIs for M
BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
Based on 10000 bootstrap replicates
```

```
# A tibble: 4 x 4
  interval_type conf_level lower upper
  <chr>          <dbl> <dbl> <dbl>
1 normal          0.95  8.02  41.3
2 basic           0.95    4    38
3 percent         0.95   16    50
4 bca             0.95   15    46
```

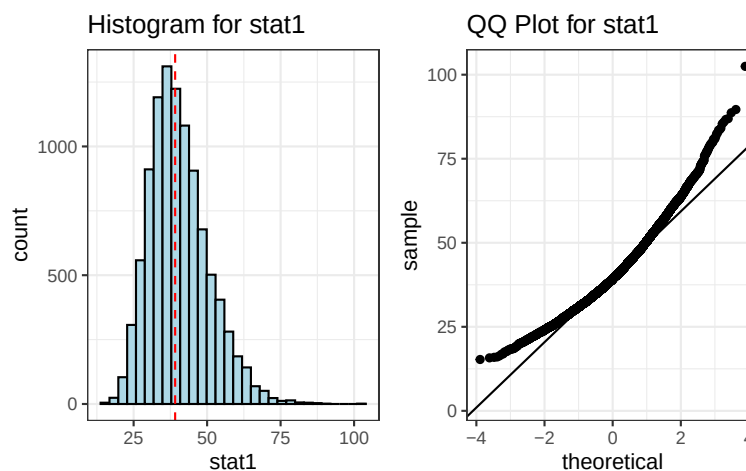
Calculations and Intervals on Original Scale
Some BCa intervals may be unstable

Hence, a 95% nonparametric CI based on this method is (\$15, \$46), which is asymmetric about the estimate of $m = \$27$. One can change the confidence level using the optional argument `conf=` for `boot_ci()`; see the **R** help menu. The BCa method is based on the actual distribution of the bootstrap estimates (rather than just their standard deviation) and requires a large number of bootstrap replications to be reliable (say 5000 or more for a 95% CI). In general, more bootstrap replications are needed to reliably estimate percentiles further in the tails (for a 99% CI, for example, where one needs the 0.5% and 99.5% percentiles).

What if we wanted a 95% confidence interval for the 10% trimmed mean grocery amount? We would carry out the same steps, but would bootstrap the 10% trimmed mean rather than the median. [A 10% trimmed mean in this case drops the three highest and three lowest values, thus avoiding the three outliers.] Code for doing this appears below with the output following. Below that are a histogram of the bootstrap trimmed means and corresponding normal quantile plot are shown above. Also, a 95% confidence interval for the true 10% trimmed mean grocery cost is reported.

```
tpct <- 10
trmean <- function(x, i, trimpct) {mean(x[i], trim = trimpct/100)}
(b3 <- bootstrapping(spent, trmean, 10000, trimpct = tpct))
# A tibble: 1 x 5
  statistic sample_average original_value bias std.error
  <chr>          <dbl>          <dbl> <dbl>    <dbl>
1 stat1          40.5            39.1  1.40     10.1
```

plot(b3)



For more on the Bootstrap, including corrections one can apply to improve these confidence intervals, please consult the book *An introduction to the bootstrap* by Efron, B., & Tibshirani, R. J.

Note that we can use ideas similar to those used in the bootstrap in cross validation: instead of dividing the data into equal partitions, we simply bootstrap many times.

More on the caret Package

We have discussed the kNN machine learning algorithm. This is just one of many algorithms out there. Many of these algorithms are implemented in **R**. However, they are distributed via different packages, developed by different authors, and often use different syntax.

The **caret** (short for **C**lassification **A**nd **R**egression **T**raining) package tries to consolidate these differences and provide consistency. It currently includes 238 different methods which are summarized in the **caret** package manual¹.

¹<https://topepo.github.io/caret/available-models.html>

Keep in mind that **caret** does not include the needed packages and, to implement a package through **caret**, you still need to install the library. The required packages for each method are described in the package manual.

The **caret train()** function

The **caret** package also provides a function that performs cross validation for us. Here we provide some examples showing how we use this incredibly helpful package. We will use the 2 or 7 example to illustrate:

```
library(tidyverse)
library(dslabs)
data("mnist_27")
```

The **caret train** function lets us train different algorithms using similar syntax. So, for example, we can type:

```
library(caret)
train_glm <- train(y ~ ., method = "glm", data = mnist_27$train)
train_knn <- train(y ~ ., method = "knn", data = mnist_27$train)
```

To make predictions, we can use the output of this function directly without needing to look at the specifics of **predict.glm** and **predict.knn**. Instead, we can learn how to obtain predictions from **predict.train**.

The code looks the same for both methods:

```
y_hat_glm <- predict(train_glm, mnist_27$test, type = "raw")
y_hat_knn <- predict(train_knn, mnist_27$test, type = "raw")
```

This permits us to quickly compare the algorithms. For example, we can compare the accuracy like this:

```
confusionMatrix(y_hat_glm, mnist_27$test$y)$overall[["Accuracy"]]
[1] 0.775
```

```
confusionMatrix(y_hat_knn, mnist_27$test$y)$overall[["Accuracy"]]
[1] 0.835
```

Cross Validation

When an algorithm includes a tuning parameter, **train** automatically uses cross validation to decide among a few default values. To find out what parameter or parameters are optimized, you can read the manual ² or study the output of:

```
getModelInfo("knn")
```

We can also use a quick lookup like this:

```
modelLookup("knn")
```

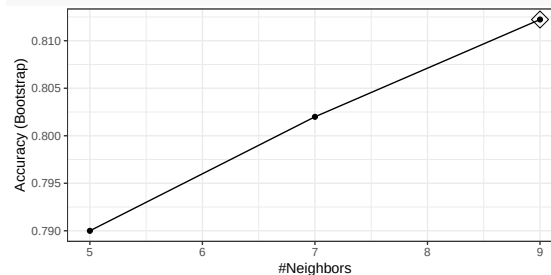
²<http://topepo.github.io/caret/available-models.html>

If we run it with default values:

```
train_knn <- train(y ~ ., method = "knn", data = mnist_27$train)
```

You can quickly see the results of the cross validation using the `ggplot` function. The argument `highlight` highlights the max (thought it may give a warning about a function being deprecated):

```
ggplot(train_knn, highlight=TRUE)+theme_bw()
```



By default, the cross validation is performed by taking 25 bootstrap samples comprised of 25% of the observations. We can change this to K-fold cross validation using the `trControl` option:

```
trControl = trainControl(method = "cv", number = 10).
```

It does 10-fold cross validation by default if `number` is not specified.

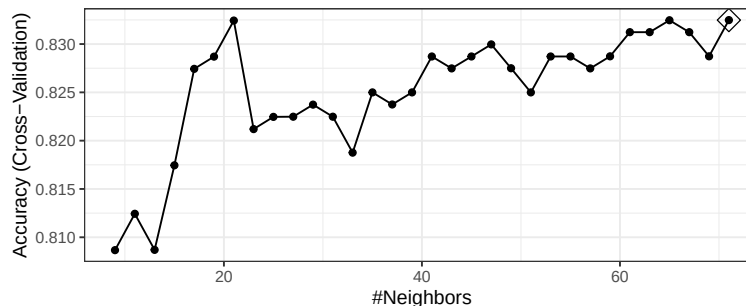
For the `kNN` method, the default is to try $k = 5, 7, 9$. We change this using the `tuneGrid` parameter. The grid of values must be supplied by a data frame with the parameter names as specified in the `modelLookup` output.

Here, we present an example where we try out 30 values between 9 and 67. To do this with `caret`, we need to define a column named `k`, so we use this: `data.frame(k = seq(9, 67, 2))`.

Note that when running this code, we are fitting 30 versions of `kNN` to 25 bootstrapped samples. Since we are fitting $30 \times 25 = 750$ `kNN` models.

We will set the seed because cross validation is a random procedure and we want to make sure the result here is reproducible.

```
set.seed(2026)
train_knn <- train(y ~ ., method = "knn",
  data = mnist_27$train,
  trControl = trainControl(method = "cv"),
  tuneGrid = data.frame(k = seq(9, 71, 2)))
ggplot(train_knn, highlight = TRUE) + theme_bw()
```



```
train_knn$bestTune
```

```
  k
32 71
```

To access the parameter that maximized the accuracy, you can use this:

```
train_knn$bestTune
```

```
  k
32 71
```

and the best performing model like this:

```
train_knn$finalModel
```

```
71-nearest neighbor model
```

```
Training set outcome distribution:
```

```
  2    7
401 399
```

The function `predict` will use this best performing model. Here is the accuracy of the best model when applied to the test set, which we have not used at all yet because the cross validation was done on the training set:

```
confusionMatrix(predict(train_knn, mnist_27$test, type = "raw"),
  mnist_27$test$y)$overall["Accuracy"]
```

```
Accuracy
0.825
```

The `results` component of the `train` output includes several summary statistics related to the variability of the cross validation estimates:

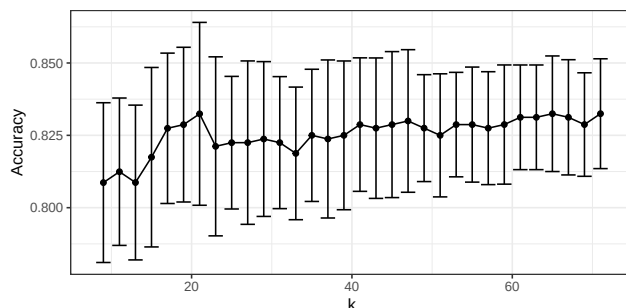
```
names(train_knn$results)
```

```
[1] "k"          "Accuracy"   "Kappa"      "AccuracySD"
[5] "KappaSD"
```

Note: `Kappa` is [Cohen's Kappa](#) and it is like classification accuracy, except that it is normalized at the baseline of random chance on your dataset. It is a more useful measure to use on problems that have an imbalance in the classes

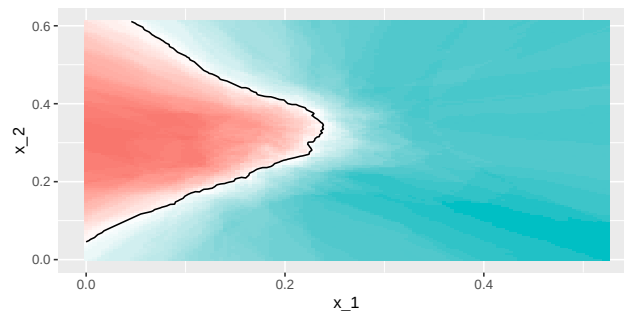
We can also see the standard deviation bars obtained from the cross validation samples. These are not confidence intervals.

```
train_knn$results |>
  ggplot(aes(x = k, y = Accuracy)) +
  geom_line() + geom_point() + theme_bw() +
  geom_errorbar(aes(x = k,
    ymin = Accuracy - AccuracySD,
    ymax = Accuracy + AccuracySD))
```



Example: Fitting with Loess

The best fitting kNN model approximates the true conditional probability:



However, we do see that the boundary is somewhat wiggly. This is because kNN, like the basic bin smoother, does not use a kernel. Kernels can be used to make a smoother line since they are based on models instead of just the empirical data.

To improve this we could try using a loess kernel. By reading through the available models part of the manual³ we see that we can use the `gamLoess` method. In the manual⁴ we also see that we need to install the `gam` package if we have not done so already with `install.packages("gam")`.

Then we see that we have two parameters to optimize:

```
modelLookup("gamLoess")
```

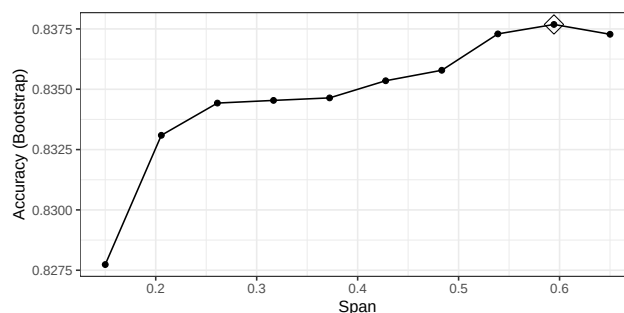
	model	parameter	label	forReg	forClass	probModel
1	gamLoess	span	Span	TRUE	TRUE	TRUE
2	gamLoess	degree	Degree	TRUE	TRUE	TRUE

We will stick to a degree of 1. But to try out different values for the span, we still have to include a column in the table with the name `degree` so we can do this:

```
grid <- expand.grid(span = seq(0.15, 0.65, length = 10),
  degree = 1)
```

We will use the default cross validation control parameters.

```
set.seed(2026)
train_loess <- train(y ~ ., method = "gamLoess",
  tuneGrid = grid, data = mnist_27$train)
ggplot(train_loess, highlight = TRUE) + theme_bw()
```



³<https://topepo.github.io/caret/available-models.html>

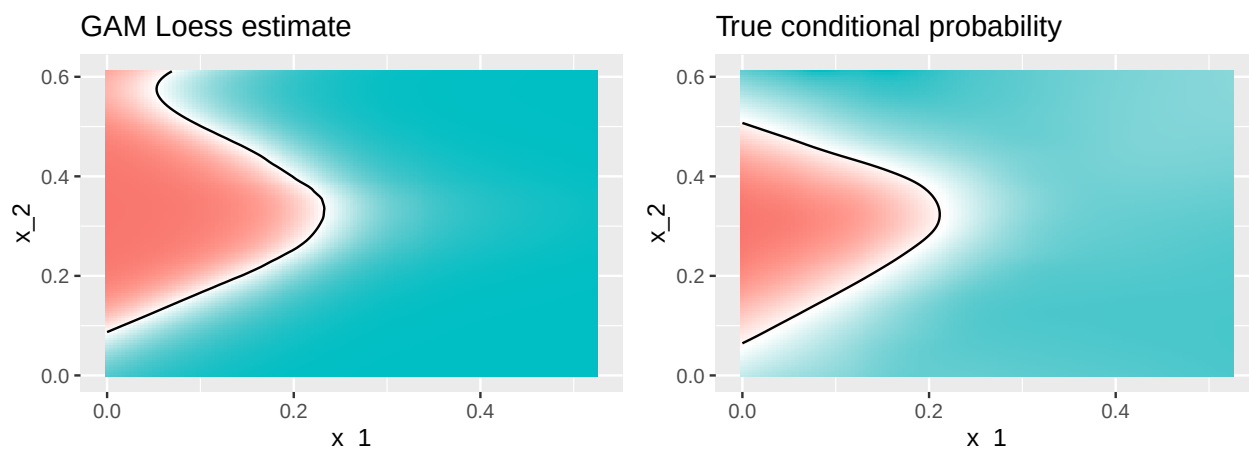
⁴<https://topepo.github.io/caret/train-models-by-tag.html>

We can see that the method performs similar to kNN:

```
confusionMatrix(data = predict(train_loess, mnist_27$test),  
                 reference = mnist_27$test$y)$overall["Accuracy"]
```

Accuracy
0.83

It produces a smoother estimate of the conditional probability, which, although not at all necessary, is a bit more aesthetically pleasing:



Conclusion

Many algorithms in statistical and machine learning rely on cross validation and/or other similar verification methods. We will continue to see this used as we continue in the course.